

CSA0918- PROGRAMMING IN JAVA CO2 AT1 SCENARIO BASED QUESTIONS

NAME:VANITHA.K

REG NO:192511027

A monitoring system must inspect a list from beginning to end and also update selected elements during traversal. Choose between Iterator, ListIterator and index-based traversal, and justify the choice

Objective

To select the most suitable traversal mechanism in Java for a monitoring system that needs to **traverse a list from beginning to end and update selected elements safely during traversal.**

Analysis

The best choice is **ListIterator**.

Method	Forward Traversal	Update During Traversal	Elements Traversal	Suitable?
Iterator	Yes	Limited (remove() only)		
ListIterator	Yes	Yes (set())		
Index-based traversal	Yes	Yes		Less flexible

Justification

ListIterator is preferred because:

1. It can traverse the list **from beginning to end** using hasNext() and next().
2. It provides the set() method to **replace/update the current element**.
3. It also supports add() and remove() when required.
4. It is specifically designed for **List implementations** such as ArrayList and LinkedList.
5. It avoids manually managing indexes, making the code clearer and safer when the list structure changes.

Pseudocode

START

Create a list of monitoring records

Create ListIterator for the list

WHILE iterator.hasNext()

 record = iterator.next()

 IF record requires update THEN

 update record

 iterator.set(updated record)

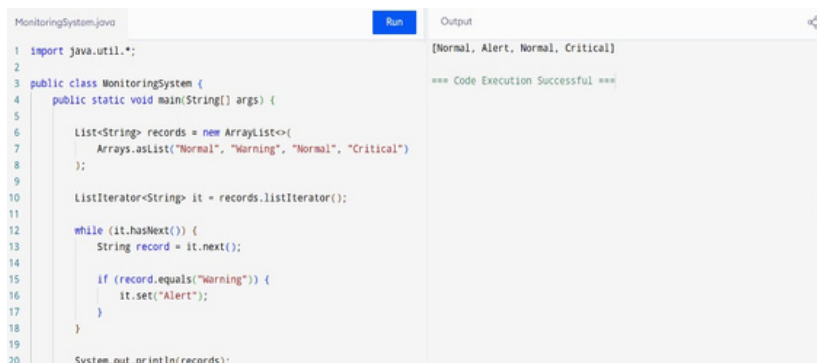
 END IF

ENDWHILE

Display updated list

END

Java Example



The screenshot shows a Java IDE with a file named 'MonitoringSystem.java'. The code defines a 'MonitoringSystem' class with a 'main' method. It creates a list of records: 'Normal', 'Warning', 'Normal', and 'Critical'. A 'ListIterator' is used to iterate through these records. When a 'Warning' record is encountered, it is updated to 'Alert'. The final list of records is printed to the console.

```
1 import java.util.*;
2
3 public class MonitoringSystem {
4     public static void main(String[] args) {
5
6         List<String> records = new ArrayList<>()
7             Arrays.asList("Normal", "Warning", "Normal", "Critical")
8         );
9
10        ListIterator<String> it = records.listIterator();
11
12        while (it.hasNext()) {
13            String record = it.next();
14
15            if (record.equals("Warning")) {
16                it.set("Alert");
17            }
18        }
19
20        System.out.println(records);
21    }
22 }
```

The output window shows the result: '[Normal, Alert, Normal, Critical]' and a message '=== Code Execution Successful ==='.

Conclusion

ListIterator is the most suitable choice because the system requires both **sequential traversal and modification of selected list elements**. Index-based traversal is possible, but ListIterator provides a cleaner and safer solution specifically designed for this requirement.